

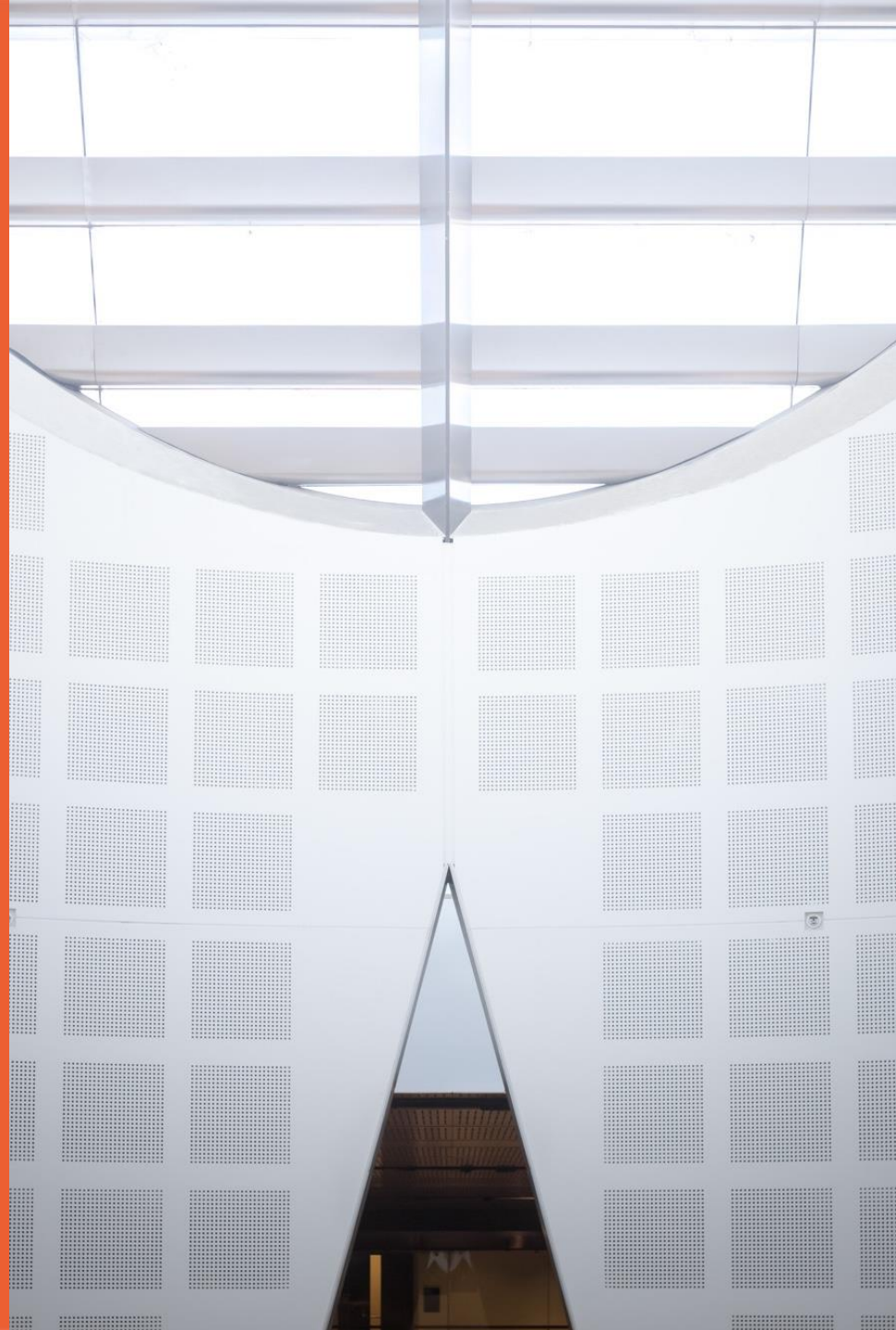
Web fundamentals & Project management

Presented by
Jiawen Wen

ELEC3609/9609
Internet Software Platforms
2026 S2



THE UNIVERSITY OF
SYDNEY



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

Web fundamentals

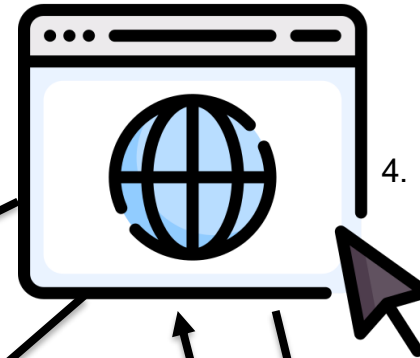
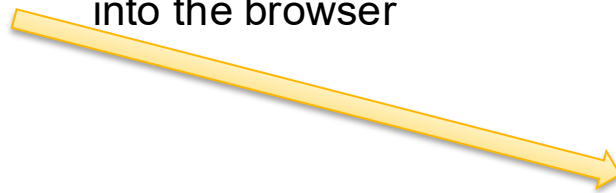
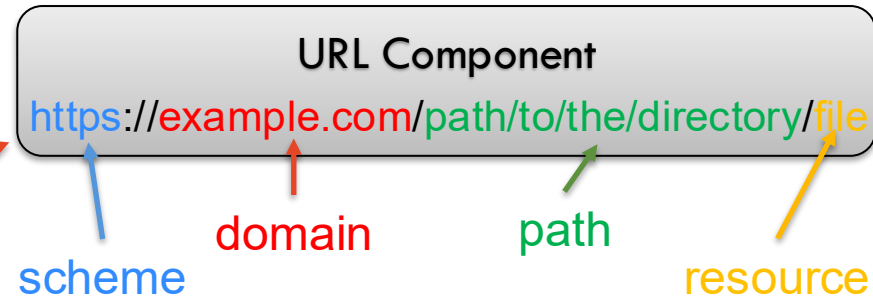
- ✓ **What happens when you enter an URL from a browser**
- ✓ **Making a web request**
- ✓ **Responding to a web request**
- ✓ **Parse a web response**
- ✓ **Web hosts**

What happens when you enter an URL from a browser?

Web request overview



Bob enters a **URL** into the browser



4. Render the content

1.1 Browser looks up IP using recursive DNS lookup



DNS Resolver

Recursive lookup



1. Browser looks up IP in cache

3. Send back the response

2. Establishes TCP connections; Send HTTPs request



Web request overview

- Client:
 - 1. Browser: Parse URL
 - 2. Browser: Check cache
 - 3. OS: Identify network
 - 4. OS: Resolve using DNS
 - 5. OS: Establish TCP connection to web server
 - 6. Browser: Submit HTTP request to web server
- Server:
 - 1. Load balancers (LB) establish TCP connection and receive HTTP request.
 - 2. LB software parses HTTP header, then routes and forwards request to an application (app) server.
 - 3. App server executes back-end website logic, probably connecting to one or more databases.
 - 4. App server renders a response, returns it via TCP connection.

Web request overview

- Client:
 1. Browser: Parse response
 2. Browser: Fetch additional resources
 - Fetch assets (CSS, JS, Images)
 - Establish web socket connection/s
 - Load browser plugins
 3. Render page

Making a web request

Overview – Paser URL

protocol://domain.tld/path/file.ext

https://cusp.sydney.edu.au/students/view-unit-page/uos_id/101257

-> https://www.sydney.edu.au/units/ELEC3609

- protocol: Hyper Text Transfer Protocol (Secure) - *HTTPS*
- domain: cusp.sydney.edu.au
 - *subdomain : cusp*
 - *domain : sydney*
 - *generic top-level domain (gTLD) : edu*
 - *country code top-level domain (ccTLD) : au*
- path: /students/view-unit-page/uos_id/101257

Check Browser Cache

There are many caching layers involved on the internet. One is the **browser cache**. It will store copies of all kinds of assets.

- *JavaScript files*
- *CSS files*
- *Images*
- *Fonts*

Updating the cache

If the browser wants to check if something in it's cache is up-to-date without downloading it again, the browser can set the HTTP header:

`If-Modified-Since`

If the content has not been modified since that date, the server will respond with the HTTP Status Code:

`HTTP 304 - Not Modified`

Otherwise, you would get the regular: `HTTP 200 - OK`

Identify Network Landscape

When you connect to a network, usually your computer is configured with the **Dynamic Host Configuration Protocol (DHCP)**.

This provides your computer with:

- **IP Address:** Identifies your computer on the local network.
- **Subnet Mask:** Defines what “local” means with regards to your IP address.
- **Default Gateway:** A host that can route packets to non-local IP addresses.
- **Nameservers:** A host which can be used to resolves domain names. A DNS server.

First thing to check if you cannot reach any web sites

Resolve IP to DNS

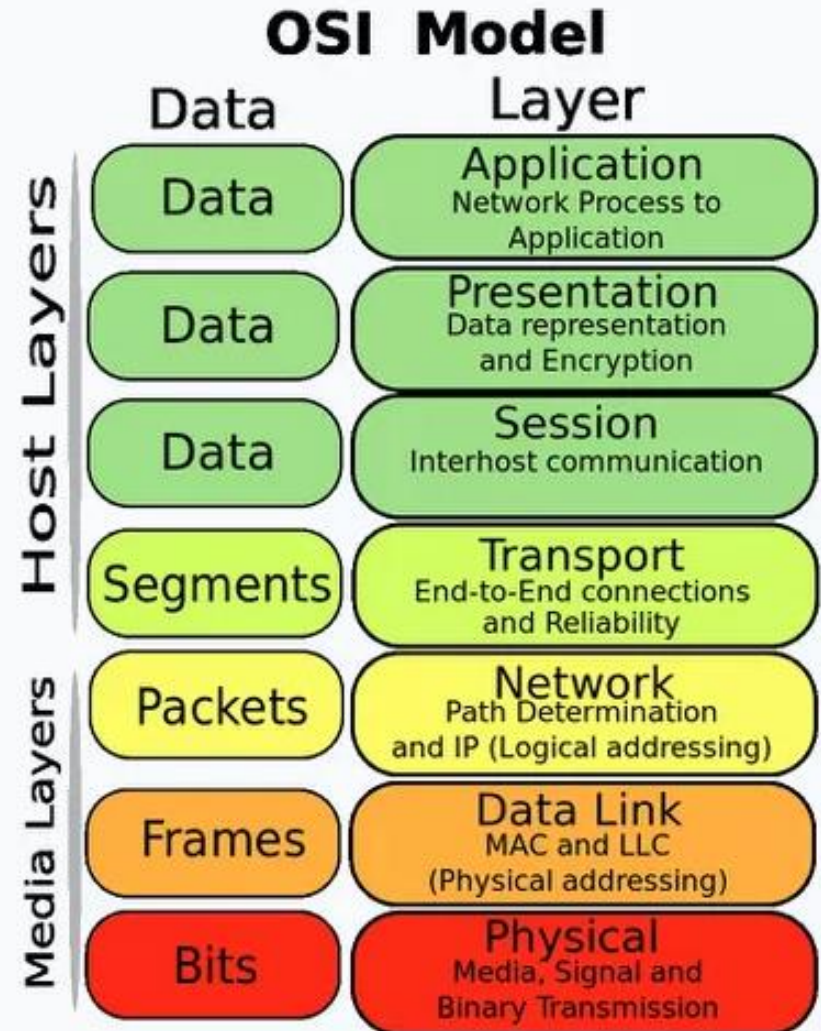
We can't establish a connection with 'cusp.sydney.edu.au'. Instead, we need an IP address.

The operating system can resolve a domain name to an IP address for us. The resolution process is roughly:

1. Check OS local DNS entries (e.g. /etc/hosts/)
 2. Identify nameservers (a.k.a DNS server)
 - Usually provided by DHCP
 - Specified in /etc/resolv.conf
 3. Submit a UDP request to a nameserver on port 53.
 4. Name servers keep recursively asking their parent name server until one knows the IP.
 5. The IP is returned according to a matching A-record for the domain.
- Check out [DNS record types](#) for more information.

Recap - Network basics: OSI model

- *Refreshing the understanding on the network basics*
- The OSI Model is a way to conceptualise network systems, whereby each layer is an abstraction from the previous.
- *Layer 1: Copper line, COAX, Fibre etc*
- *Layer 2: IEEE 802.11a/b/g/n*
- *Layer 4: TCP/UDP*
- *Layer 7: HTTPS*



Recap - TCP / UDP

	TCP	UDP
Network Layer	4	4
Mechanisms	<i>TCP guarantees in-order delivery of data for the application layer(s) above it.</i> <i>Hand shake.</i> <i>Famous cyber attack pattern: TCP flooding</i>	<i>UDP is a “best effort” protocol, which sends “datagrams” instead of TCP’s “segments”.</i> <i>With UDP, packets are not acknowledged or re-transmitted (unless performed explicitly at the application layer). No hand shake enforced</i>
Port designation	<i>Needed</i> <i>e.g. SSH on TCP/22</i>	<i>Needed</i> <i>e.g. DNS on UDP/53</i>

Recap - Establish TCP Connection

Once we have an IP address, we must open a TCP connection.

The Transmission Control Protocol (TCP) lives on layer 4 of the OSI Model. It is not responsible for addressing or segregation like the layers below it, but is instead responsible for delivery of packets.

TCP guarantees in-order delivery of data for the application layers above it.

TCP Segments are assigned to particular 'ports', which define what application they are intended for.

Depending on which protocol we use, a standard TCP port is chosen by default:

- **HTTP:** Port 80
- **HTTPS:** Port 443

(Web servers can run on non-standard ports, these are just conventions)

Recap - Aside: User Datagram Packet

The User Datagram Protocol (UDP) also lives on layer 4 with TCP, however it does not guarantee delivery at all.

UDP is a ‘best effort’ protocol, which sends ‘datagrams’ instead of TCP’s ‘segments’.

With UDP, packet deliveries are not acknowledged, and packets are not re-transmitted (unless performed explicitly at the application layer).

UDP also operates on a particular “port”, though a TCP and UDP port are considered different.

DNS operates on UDP port 53 and it does not use TCP.

HTTP Structure

- A HTTP request or responses consists of:
 - **Header:** Contain meta-data such as cookie and connection information.
 - **Body:** Contains the data to be sent with the transmission (optional).
- There are several types of web requests:
 - **GET:** A simple request header only
 - **POST:** A request containing data in the body, e.g. form data, file upload, etc.
 - **HEAD:** The same as GET, except server omits the body in the response (only sends header)
 - **PUT:** Replace a target resource on the server
 - **DELETE:** Delete a target resource on the server
 - **OPTIONS:** Ask the server which request types are allowed against a particular resource

HTTP request example

Now that we have an open TCP connection to the web server we just need to send it a HTTP request through that connection.

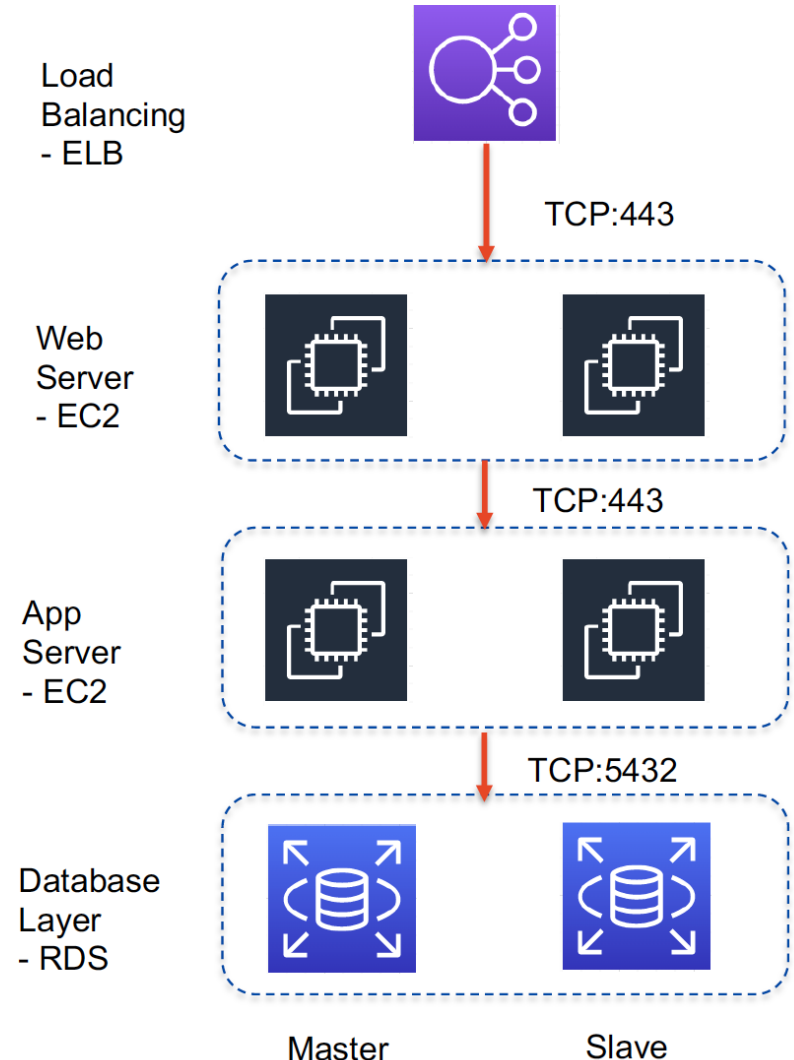
A HTTP Request

```
GET /students/view-unit-page/alpha/ELEC3609 HTTP/1.1
Host: cusp.sydney.edu.au
Connection: keep-alive
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/49.0.2623.87 Safari/537.36
Accept-Encoding: gzip, deflate, sdch
Accept-Language: en-AU,en;q=0.8,en-GB;q=0.6
Cookie: km_ai=IaRyuapbUDqQ%3D; km_lv=x; km_uq=; BIGipServer~APPLICATION~cusp.sydney.edu.au_443=rd1o000000000000000000000000ffffac140537o80; _jsuid
```

Responding to a web request

Typical architecture – three layers

- Use AWS IaaS service as an example.
 - End-to-end encryption in transit
 - Elastic Load Balancing (ELB)
 - Elastic Cloud Compute (EC2)
 - Relational Database Services (RDS)
- Load balancing
 - Route traffic based on performance and health status
 - Route traffic based on contents
 - SSL offloading (HTTPS)
 - Port forwarding (HTTP to HTTPS)
- Web Server
- App Server
- RDS
 - Master / Slave configuration for high availability (HA)



Internet-facing web servers

- An internet-facing web server can directly receive HTTP requests from clients. Usually, the first recipient of the HTTP request is not the service that provides the response.
- Instead, it is often:
 - A load balancer (LB) routes requests to appropriate servers for processing.
 - Load balancers allow a ‘pool’ of servers to handle incoming requests by forwarding a request to an available resource.
 - DDoS mitigation services (e.g. web application firewall) often serve as the first-layer for handling requests on large websites.

Their purpose is to simply ensure that incoming requests aren’t part of a cyber attack.

 - Legitimate packets are forwarded (often to the regular LB)
 - Attack packets are dropped
 - E.g. [CloudFlare](#)
 - Content Delivery Network (CDN)
- Secured HTTP = HTTPS by using TLS protocols (SSL is obsolete)

Application servers

- **App servers** do the actual processing of HTTP requests. They contain:
 - Back-end website code
 - Business logic
 - Database interface methods
 - Caching layers
- Typical form of web servers:
 - Virtual machines (IaaS): EC2, GCE
 - Containers: docker
 - Serverless: Lambda, Google Cloud Function
- This is where the majority of the work lies in a typical web architecture.

App Server programming models

- The app server could be written in many different languages. Since there are lots of standard web-server functions, developers choose
- To work within a web framework rather than re-inventing the wheel.
To name a few languages and some associated web frameworks:
(subject to changes without notice)
- Python (e.g., Instagram)
 - [Flask](#)
 - [Django](#)
- PHP (e.g., Facebook)
 - [Codeigniter](#)
 - [CakePHP](#)
- Ruby (e.g., GitHub)
 - [Ruby On Rails \(RoR\)](#)
- JavaScript (e.g., Groupon)
 - [Express.js \(on Node.js\)](#)

Database – persistent data

- The app servers typically read information from a databases.
- There are many types and flavours of database.
We will split them into break them into two main types:
- SQL: Relational databases maintain and enforce relationships between data.
 - MySQL
 - PostgreSQL
 - SQLite
 - Microsoft SQL Server
- NoSQL: Non-relational databases store data more like loose pages, and often without no schemas.
 - MongoDB
 - Redis
 - DynamoDB, Cloud Data Store
- Databases can reside on the same host as the app server, or separately (Recommended)

Slave/Master Databases

When handling lots of web requests, we must read and write to the database frequently.

To spread the load on the database, we separate read and write operations to slave and master DBs respectively:

Master/Primary: The master database accepts reads and **write** operations. Since it is difficult to have multiple write locations, there is usually only **one master** database.

Slaves/Replicas: Any updates to the master database are quickly replicated to all slave databases.

- Slaves can be used for **read operations only**.
- Since data is not modified on slaves, there can be many replicated slaves, improving parallelizability of DB reads.

Slave Drift

- Any changes applied to the master DB will also be replicated to the slave database.
 - However, this does not occur instantaneously and there is a small, but significant, **time delay** between a write to the master DB and the propagation of the change to all slave DBs.
- This is called **slave-drift** or **replication delay** and must be managed appropriately by your application.
- DB reads that lead to important decisions, such as payments, should usually be made against the master to prevent race conditions or side-effects by reading potentially stale data.
- There are replication configurations that avoid this problem but come with other trade-offs, such as delayed writes and/or reduced fault tolerance.

Generating a HTTP response

Once the application server has performed all the business logic to decide how it will respond, it must **generate** the response.

Typically, this involves insertion of data into a **template**, which forms the HTTP response.

Jinja2 HTML Template

```
<title>
  {% block title %}{% endblock %}
</title>
<ul>
  {% for user in users %}
    <li>
      <a href="{{ user.url }}">
        {{ user.username }}
      </a>
    </li>
  {% endfor %}
</ul>
```

The variables in the application code are inserted in between {{ and }}. Code can also be specified between {% and %}

Returning a HTTP Response

Similarly, to a HTTP request, a HTTP response also contains two sections:

Header: Contains meta-data such as HTTP status code, content type, length.

Body: Typically contains the HTML, or contents of a requested resource (optional).

A HTTP response header

```
HTTP/1.1 200 OK
Date: Fri, 29 Jul 2016 04:43:00 GMT
Server: Apache/2.2.15 (Red Hat)
X-Powered-By: PHP/5.3.3
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post
               -check=0, pre-check=0
Pragma: no-cache
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8
```

Parse a web response

HTTP status code

- The response can specify a HTTP status code which indicate the status

HTTP code category	Definition	Example
1xx	Informational	100: Continue 101: Switching Protocols
2xx	Success	200: OK – the standard response
3xx	Redirection	301: Moved Permanently 302: Found/Moved Temporarily (similar to 303 & 307) 304: Not Modified
4xx	Client	404: Not found
5xx	Server	502: Bad gateway

Client Rendering

The HTTP status codes instruct the client on how to handle the response.

For example, a 302 code (Found, previously Moved Temporarily) will cause the browser to immediately follow the URL specified in the *Location* field in the response *header*.

HTTP 302 Response

```
HTTP/1.0 302 Found
Location: https://cusp.sydney.edu.au/students/view-unit-
        page/alpha/ELEC3609
Server: BigIP
Connection: Keep-Alive
Content-Length: 0
```

A 200 (OK) response is usually accompanied by a resource in the response *body*, which is rendered by the browser.

Parse HTML into DOM

- If a HTTP response contains HTML, the browser parses this HTML to build something called the Document Object Model (DOM).
- The DOM forms the browser's internal representation of the web page.
- The relationships between various HTML elements (such as `<div>` or `<img>`) tags are defined and can be manipulated via the DOM.
- It is common for JavaScript libraries, such as jQuery, to manipulate the DOM so as to modify the layout of web pages.
- Browser plugins often interact with browsers at this level

DOM Resources

While the Hyper Text Markup Language (HTML) specifies how components of the web page are structured, there are usually many other resources that define how the page behaves.

For example:

- **CSS:** Cascading Style Sheets (CSS) are used to specify positional and appearance attributes of DOM elements. CSS can be specified inside HTML `<style>` tags, or in external files that are included by means of `<link>` HTML tags in the `<head>` section.

```
<link rel="stylesheet" href="/styles/cusp_style.css" type="text/css" />
```

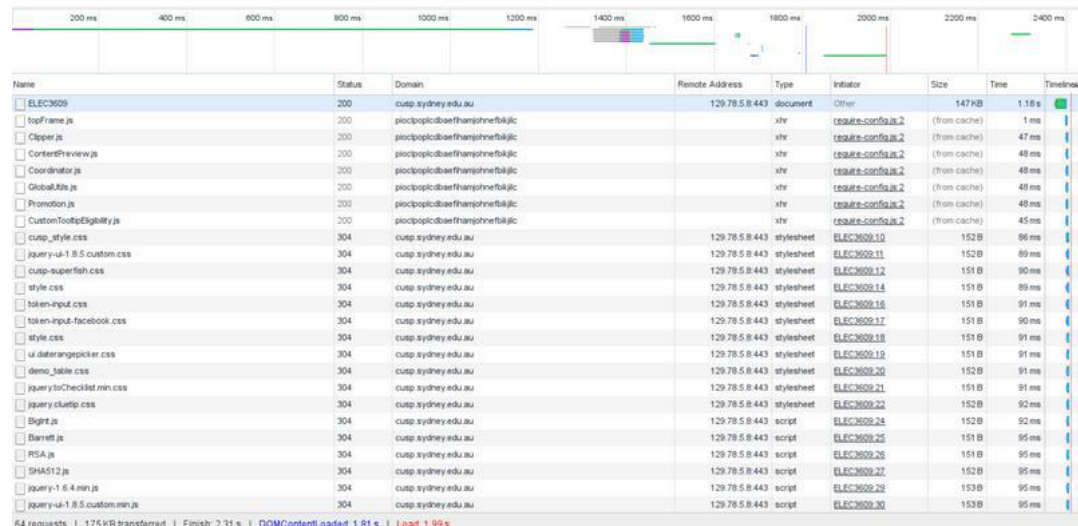
- **JS:** JavaScript (JS) can be used in the browser to perform arbitrary computation, often including live manipulation of DOM elements. JS can be specified inside HTML `<script>` tags directly, or included from an external file using the „src“ attribute.

```
<script type="text/javascript" src="/scripts/util/Util.js"> </script>
```

Fetching DOM Resources

The browser must perform many subsequent HTTP (or other) requests to the web server in order to collect these additional resources.

These resources may be on the same server as the web server, or they could be on different servers.



Browser Plugins

- It is worth mentioning that plugins in your browser can interfere with and/or modify the response.
- For example, AdBlock, prevents ads from showing.



The AdBlock browser plugin, alongside several others.

Browser rendering

- The ‘rendering engine’ or ‘web browser engine’ actually renders the page to the graphical content that you see.
- They are responsible for the look and feel of how a particular browser shows web pages.
- For example, they will decide how things like check-boxes on forms will be displayed. Some examples include:
 - **Blink**: Used in Google Chrome
 - **EdgeHTML**: Used in Microsoft’s ‘Edge’ browser
 - **WebKit**: Used in Apple’s Safari
 - **Gecko**: Used by Mozilla’s Firefox

Web Hosts

Web server software

- There are many web server software packages.
- Some of the common ones are:

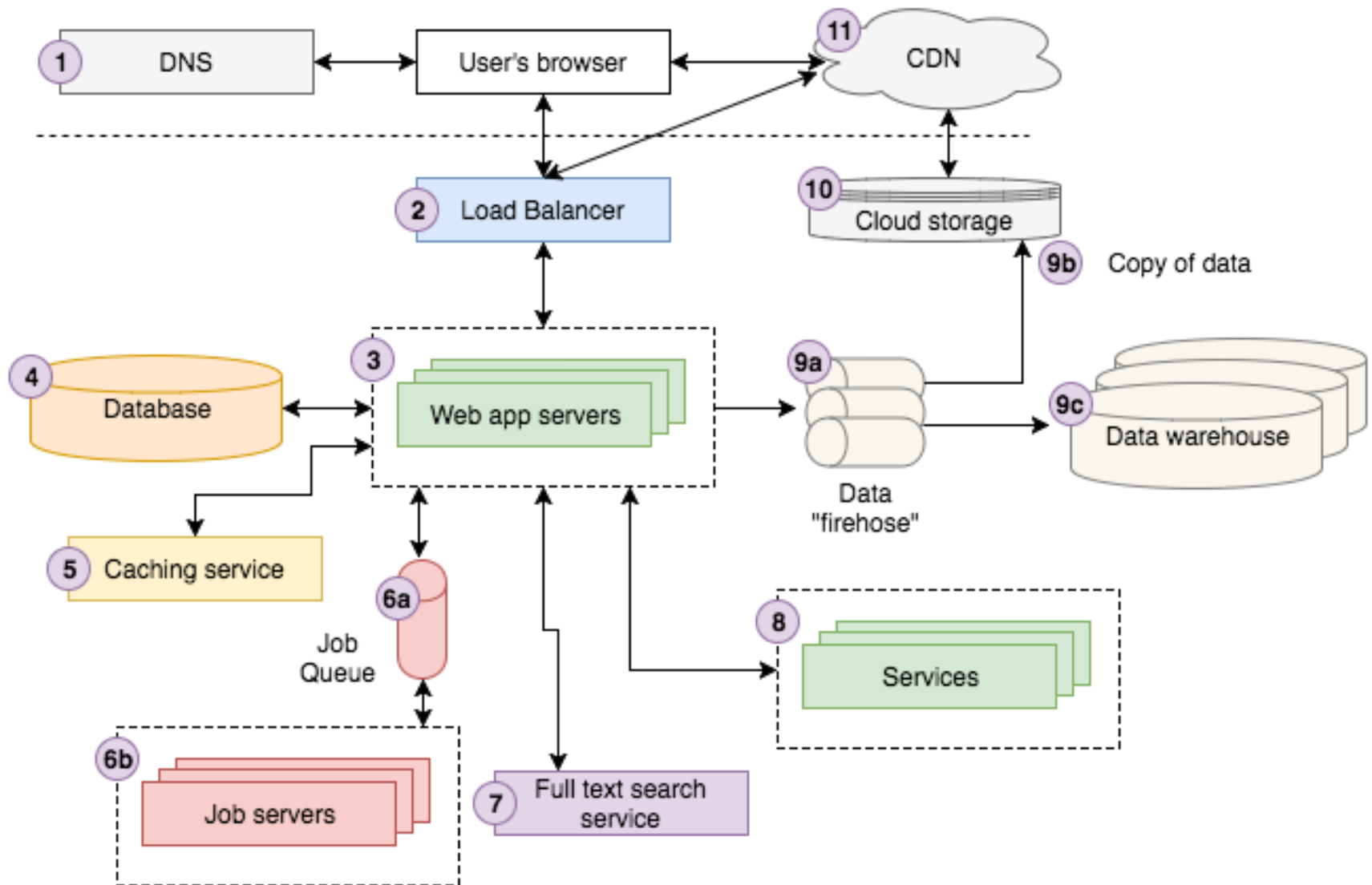
Web Server	Description
Apache	<i>Pretty simple, good for smaller websites. Typically spawns a whole new process to handle each request, which has a lot of overhead.</i>
Nginx	<i>While technically a Reverse proxy, Nginx is one of the most common web servers on the internet. It has a series of worker processes that handle incoming requests, which makes it a lot more scalable.</i>
IIS	<i>Internet Information Services is Microsoft's web server. It has a long history of security vulnerabilities, and should Be avoided if possible.</i>

Server Operating System

- Linux or Unix-like OSes make up over 60% of all server OS.
- Windows server makes up less than 40%
- Top 100 supercomputers all use Linux or Unix

Common Stacks

- LAMP
 - Linux
 - Apache
 - MySQL
 - PHP
- MEAN
 - MongoDB
 - Express.js
 - Angular.js
 - Node.js
- LYME
 - Linux
 - Yaws
 - Mnesia or CouchDB
 - Erlang



Overview – Project management

Requirement Analysis and Design

- ✓ **Introduction**
- ✓ **Use cases**
- ✓ **Sequence Diagrams**
- ✓ **State & Flow Diagrams**
- ✓ **Wireframes**
- ✓ **Documentation**
- ✓ **Task Management**

Introduction

Project Management Tools of the Trade

Written: It's important to use tools which enable collaboration and produce professional looking documents.

- Google Docs
 - allows easy collaboration
 - supports versioning
- Latex
 - looks professional (with the right template)
 - easy to use with version control systems (e.g., git)
 - great for formatting and consistency
- Microsoft Word
 - feature rich, but difficult to manage version control
 - difficult to collaborate

Project Management Tools of the Trade

Graphical: Should be able to easily make sensible and accurate diagrams.

- LucidChart
 - Multiple editors, can be integrated with Google Docs.
- Adobe Illustrator
 - Super flexible, but not as easy for a beginner to use.
- Balsamiq
 - Wireframing only
- Diagrams.net (previously Draw.io)
 - Microsoft Visio
 - Free. SaaS offering

Project Management Tools of the Trade

Management: Tools for managing workflow and development processes.

- Github
- Jira: Integrated well with software development life cycles
- Phabricator
- Asana: Light-weight web-based tool
- Microsoft Project: Traditionally used by many big projects and organisations

Use Cases

Step One: Identify Problems

What are the core issues you are trying to solve?

- Identify reasons why someone would want to use your website/solution.
- What problem does your website solve for people?
- Create a series of problem statements.
 - ‘Buyers have no way of expressing how much they want to pay for an item’
 - ‘Sellers have difficulty determining the price to sell at’
 - ‘Non-technical people have no way of getting a website built’

Step Two: Determine Use Cases

Attempts to convert your problem statements into use cases. Use cases are made up of **actions** which are performed by **actors**.

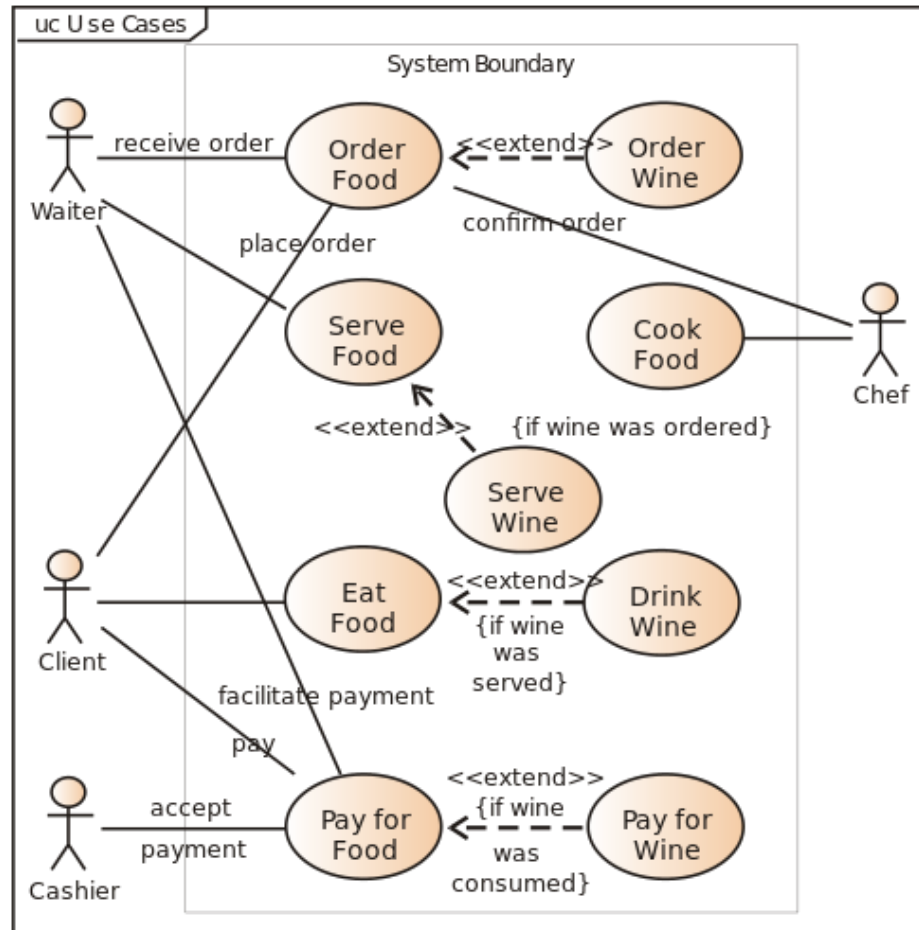
Action: A simple, distinct, and easily defined action that can be performed by an actor.

Actor: A *role* within the system, not an individual.

1. Identify flow of events
2. Extract distinct actions
3. Group associated actions into a high-level use case
4. Expand use cases into extensions

UML Use Case Diagram

UML: Unified Modelling Language



Use case model of a restaurant business

Source: [Wikimedia](#)

Use Case Documentation – Flow of Events

Create a big table that lists the use cases (see Monopoly example on Canvas).

Describe each use case using fields such as:

Name: Give the use case a short name.

Goal: The main goal that the use case is trying to achieve.

Actors: Actors which initiate a use case and/or are involved in a use case.

Preconditions: A list of conditions that must be met before the use case can start.

Main flow: A full description of the use case.

Subflows: A list of actions or use cases on which the use case depends.

Alternative Flows: Any alternative action flows, e.g. ones that might come from error scenarios.

Priority: An indication of how important the use case is to the functioning of the system, or the MVP.

Use Case Documentation – Flow of Events

X Flow-of-Events for the <name> Use Case

X.1 Preconditions. *What needs to happen (in another use case) before this use case can start? What state must the system be in before the use case?*

X.2 Main Flow. *The main flow is a series of declarative steps.*

X.3 Sub-flows. *Sub-flows break down the main flow and other sub-flows to improve document readability.*

X.4 Alternative Flows. *The alternative flows define exceptional behavior that can interrupt the normal flow. Often alternative flows indicate what is to be done under error conditions. To determine alternative flows, ask yourself, “What could possibly go wrong?” for each of the actions in the main flow and the sub-flows.*

Note: X is a unique identifier for each use case.

Figure 4: Use Case Flow-of-Events Template

Further Reading: Monopoly Example

A detailed breakdown of use cases and requirements is available in Canvas.

Monopoly Requirements Specification

Page 11-19

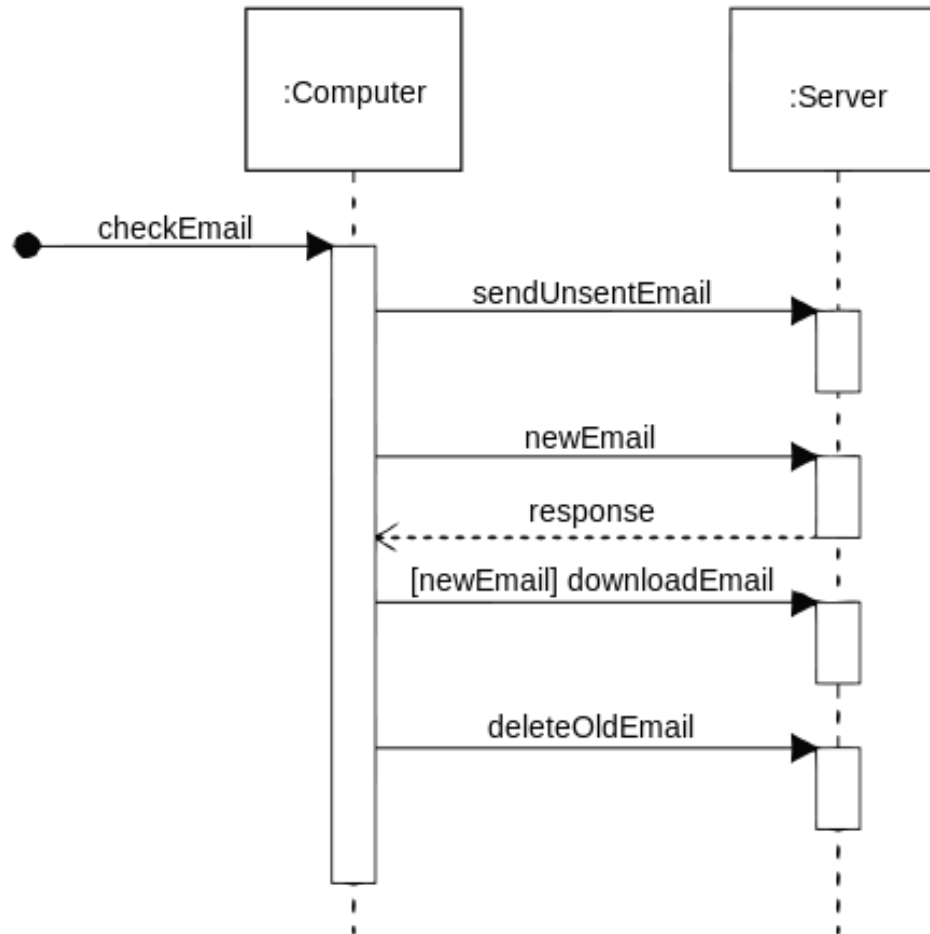
Available within Canvas: UseCaseRequirements.pdf

Sequence Diagrams

Sequence Diagrams Overview

- Sequence diagrams describe how objects interact with each other and in what order.
- Sequence diagrams are very useful for communicating how several systems interact, or providing detail around how various actors interact with system components in a multi-step process.

Sequence Diagrams Example

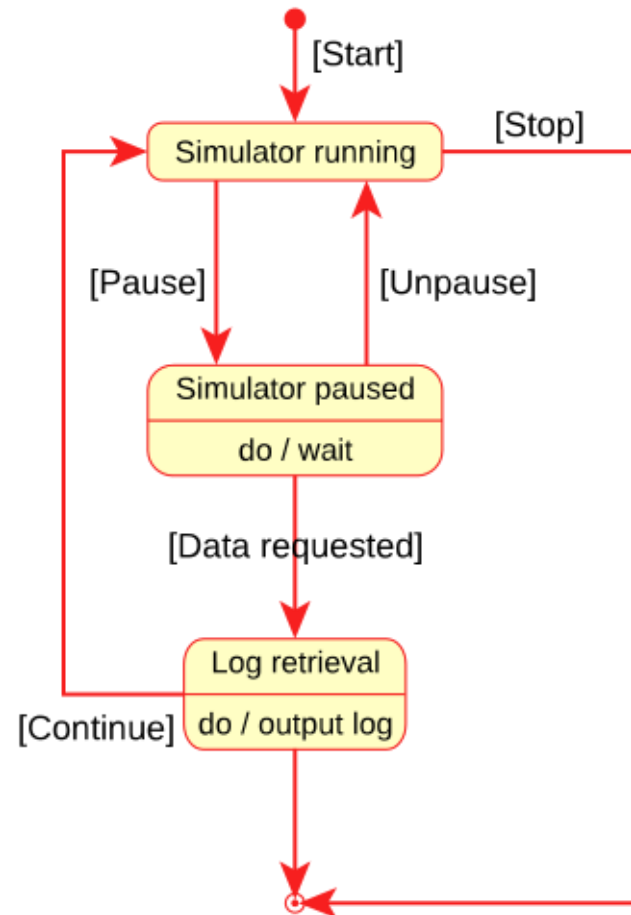


Example sequence diagram that shows a computer asking a server for emails
Source: [Wikimedia](#)

State & Flow Diagrams

State Diagrams

- As the name suggests, state diagrams (a.k.a state machine diagrams) describe the various states of system components.
- They are essential for communicating how abstract concepts such as tickets or user accounts, progress through the system.
- States themselves are defined, as are state transitions, which constrain how objects are allowed to transition between states.



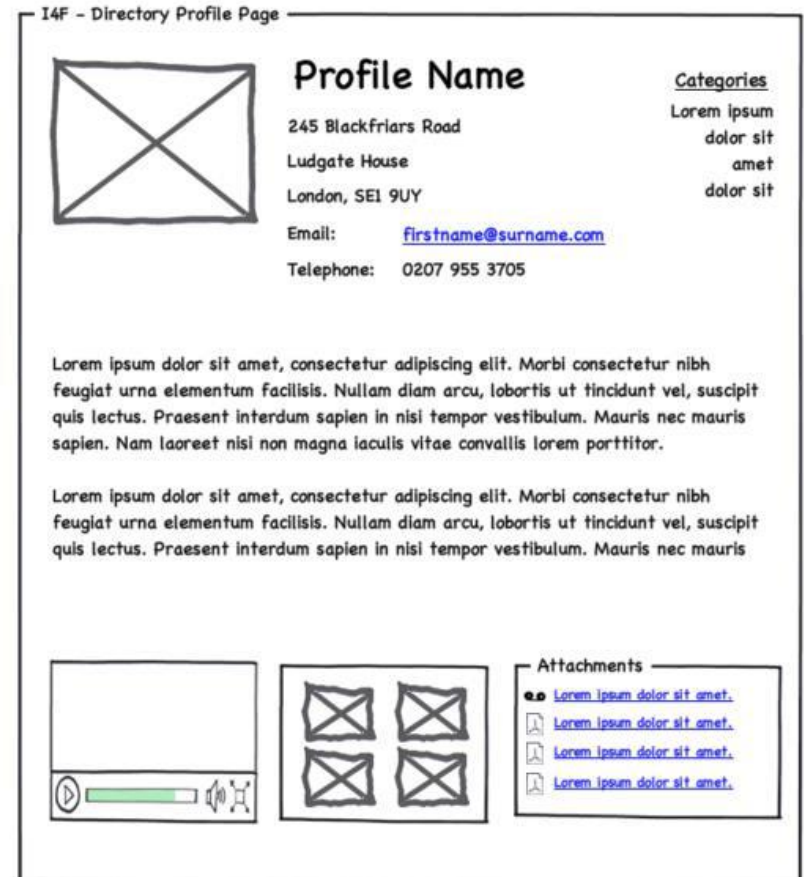
UML State Diagram example

Source: [Wikimedia](#)

Wireframes

Wireframes

- Wireframing allows designers to gain a visual ‘feel’ for the site, without focussing on graphical detail.
- The idea is to create a ‘cartoon’ version of the website that enables a preliminary website experience.
- Wireframes are usually just a series of images, but can sometimes include clicking and linkage between the images in the form of a storyboard.



created with Balsamiq Mockups - www.balsamiq.com

Profile Page Wireframe
Source: [Wikimedia](#)

Documentation

Software Requirements Specification

- A Software Requirements Specification (SRS) document details exactly what the system does and does not do.
- In essence, the SRS is a communication tool between project stakeholders and software designers.
- It is a very important document that allows you to say the project is ‘finished’, and mitigates scope creep. This forms the primary specification document and should capture all information regarding the project.
- Any diagrams should not introduce new information that is not contained within the SRS. Diagrams should provide additional context/clarification only.

SRS: Introduction

The **introduction** should encapsulate

- enough information for a business
- executive to gain an understanding of what the project is about, including all important considerations, without providing too much depth.

Introduction

- Purpose
- Definitions
- System Overview
- References

SRS: Overall Description

- The **overall description** section should discuss the project from a high level, but still examine the details such as:
 - how it's used
 - how it works
 - the scope of the system
 - dependencies and requirements
 - constraints

Overall Description

- Product Perspective
 - System Interfaces
 - User Interfaces
 - Hardware/software/communication interfaces
 - Memory constraints
- Design Constraints
 - Operations
 - Site adaption requirements
- Product functionality
- User characteristics
- Constraints, assumptions and dependencies

SRS: Specific Requirements

- The **specific requirements** section needs to get down to the nitty-gritty details.
- This section should contain highly granular requirements.
- It should be easy to identify whether a system does or does not satisfy these requirements.

Specific Requirements

- External interface requirements
- Functional requirements
- Performance requirements
- Logical database requirements
- Software system attributes
 - Reliability
 - Availability
 - Security
 - Maintainability
 - Portability

Task Management

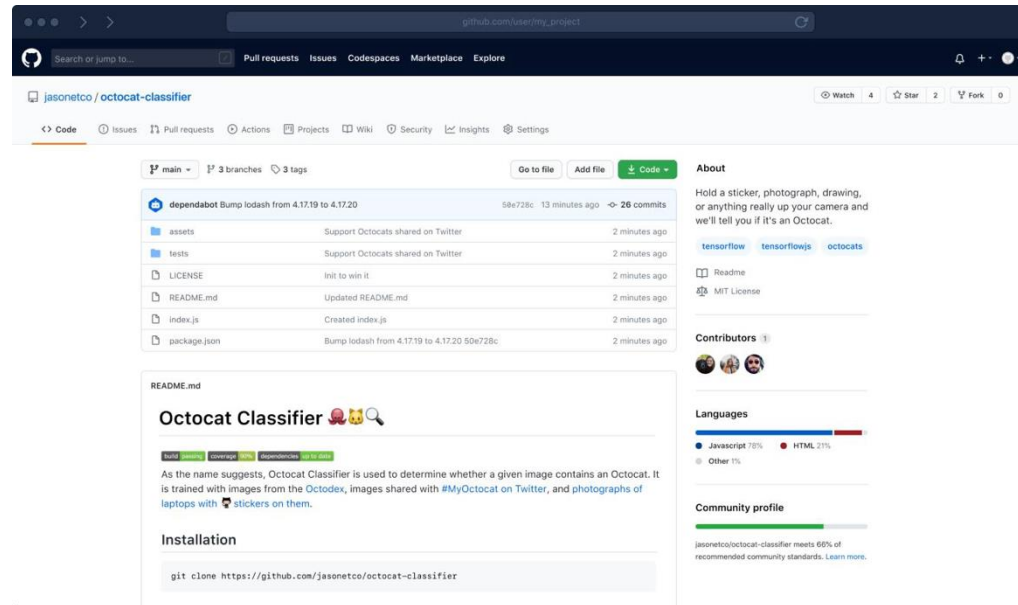
Task Management

- Building a website is hard!
- There are lots and lots of tasks to complete.
- How do we make sure that everyone knows what they are doing, and that we aren't stepping on each other's toes?

More Software!

Task Management Software – GitHub

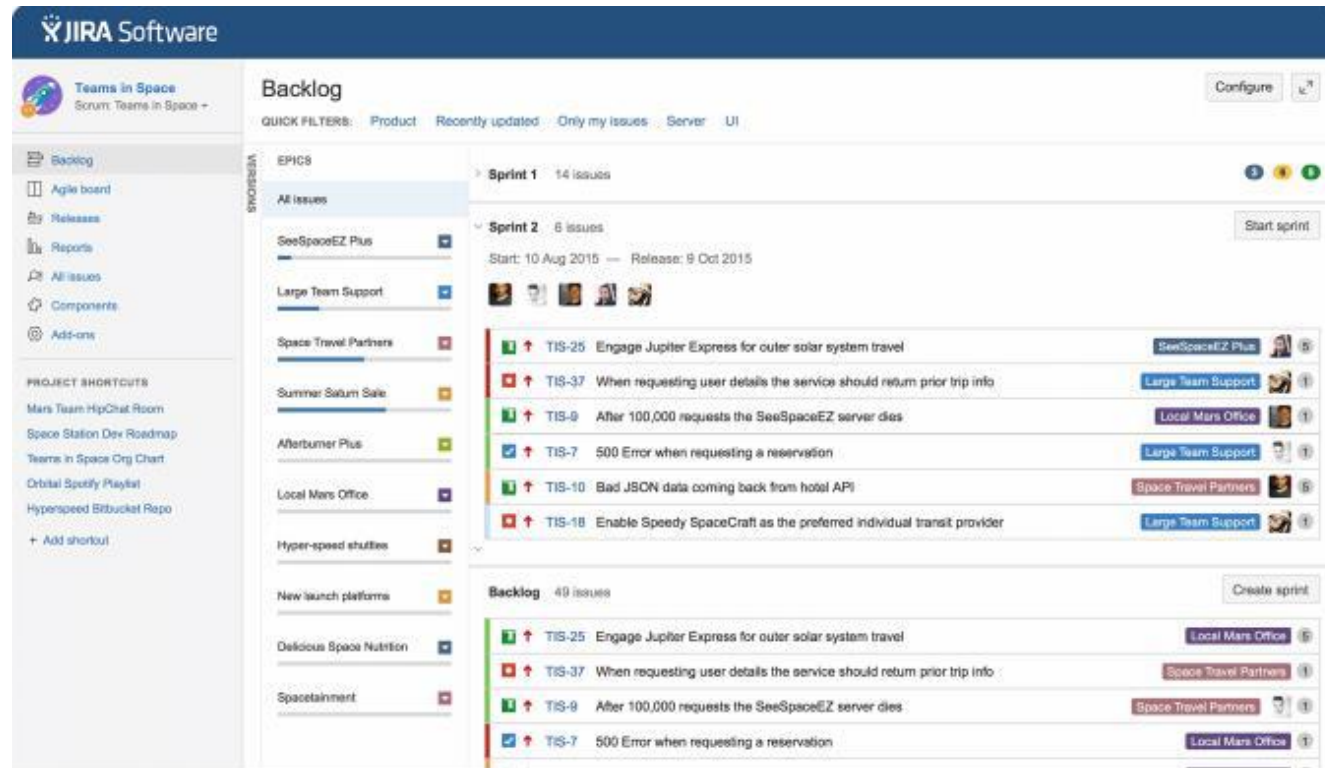
- For this course we'll be primarily using GitHub as our task management software.



Features

- Most popular source control (software configuration management tool)
- Integrated with task management – tickets / issues / defects
- Code review / scan; Security & compliance
- Platform deployment

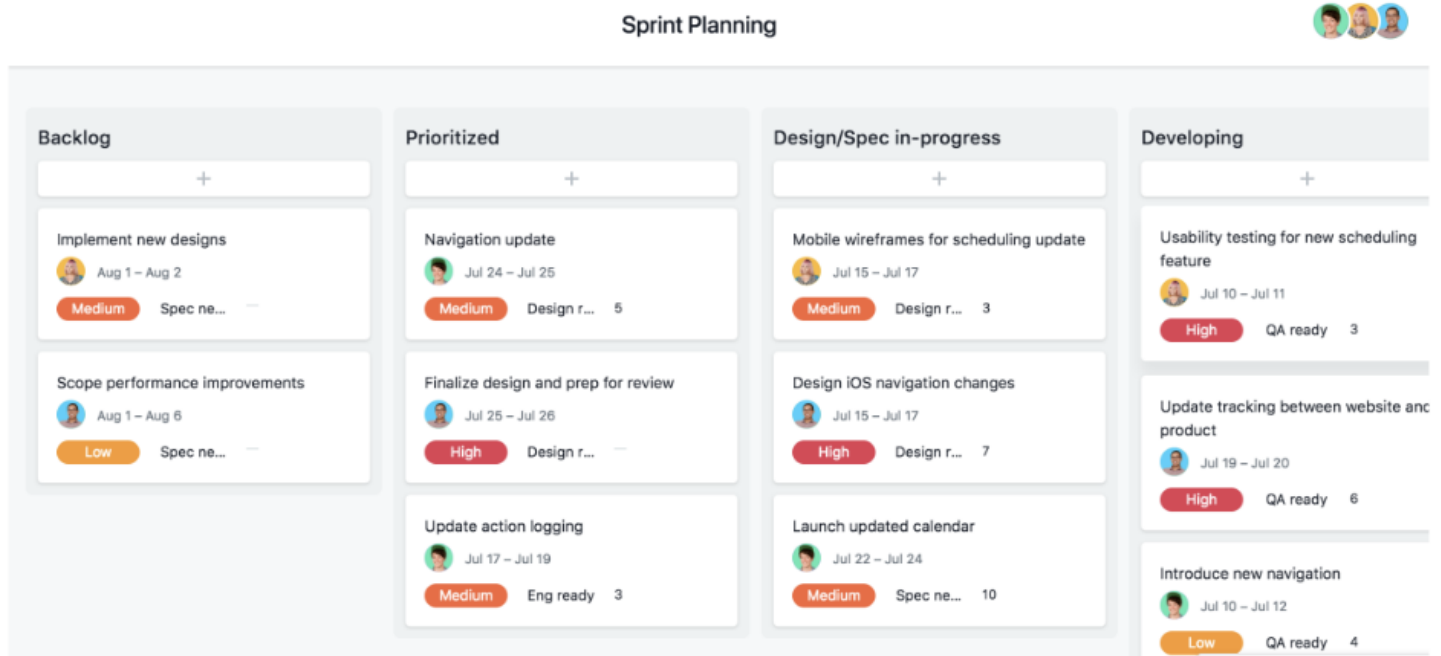
Task Management Software - JIRA



Features:

- Bug/Issue tracking, tasks
- Service tickets, timesheets
- Extendible through a marketplace/ecosystem

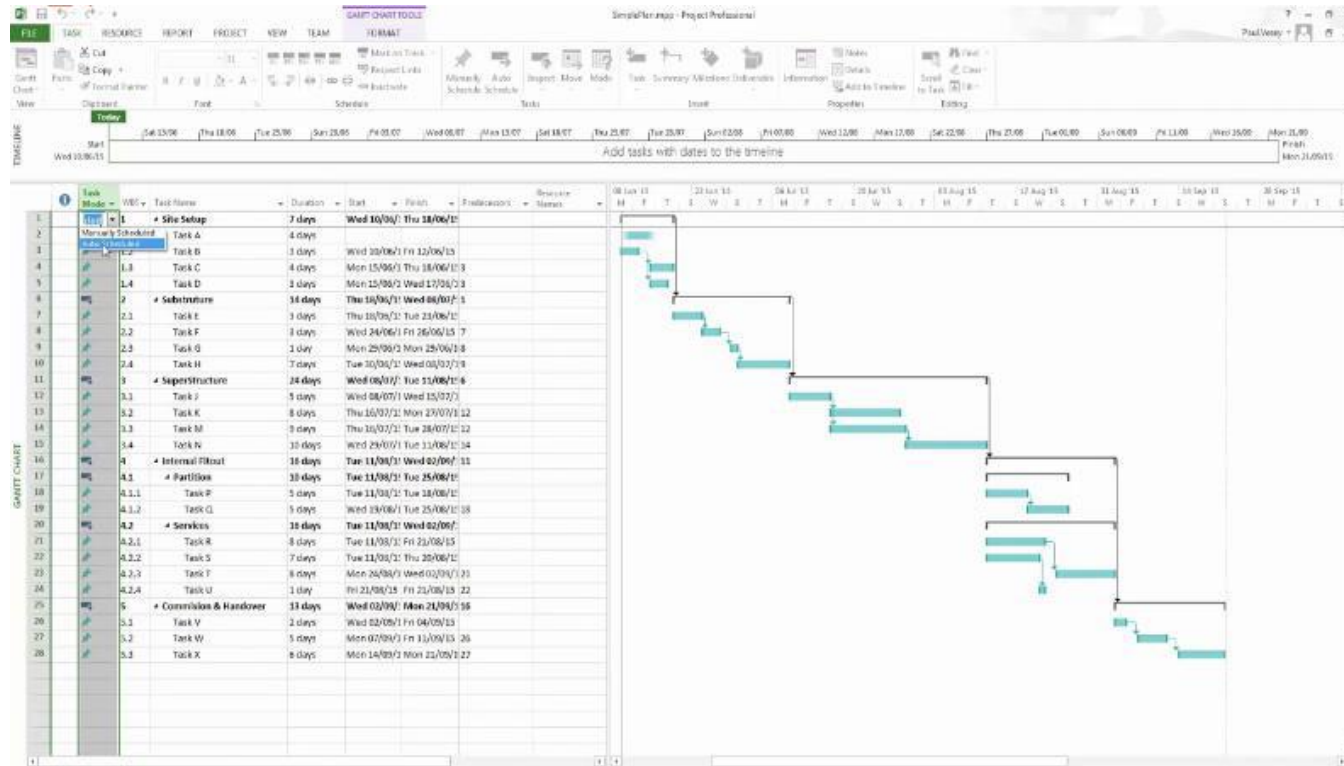
Task Management Software - Asana



Features:

- Tasks are visualised and categorised as tiles
- Similar tool: Trello

Task Management Software – Microsoft Project



Features:

- Gantt chart
- Integration with Microsoft Office
- Similar tool: SmartSheets

**You reached the end of the lecture.
Thank you and see you next week.**